Sample VAPT Report

Web & API Security Assessment

Manual SaaS · API Security · Authorization Review

Field	Details
Target	Demo SaaS Platform
Classification	Sample Report — Fictional Application
Report Version	3.0
Report Date	May 2026

Prepared by:

RedSpire Security
Founded by Harshit Shah · Cybersecurity Consultant
harshit@redspiresecurity.com
+91 94622 55272
https://redspiresecurity.com/

Disclaimer

This is a **sample report** created for illustration purposes only. All targets, application names, findings, URLs, data, and scenarios described in this document are entirely fictional. Any resemblance to real companies, applications, or vulnerabilities is coincidental.

This report demonstrates the structure, depth, and quality of reporting delivered during a real engagement. Actual engagement reports are delivered under a mutual non-disclosure agreement and treated as strictly confidential.

This document does not constitute legal, compliance, or certification advice. Both parties in a real engagement may seek independent legal counsel if required.

Document Control

Field	Details
Document Title	Sample VAPT Report — Web & API Security Assessment

Field	Details
Target	Demo SaaS Platform (Fictional)
Assessment Type	Web & API Security Assessment
Classification	Sample — Fictional / Illustration Only
Report Version	3.0
Report Date	May 2026
Prepared By	RedSpire Security — Founded by Harshit Shah · Cybersecurity Consultant
Report Status	Final (Sample)

Revision History

Version	Date	Description
1.0	May 2026	Initial sample report
2.0	May 2026	Revised to Critical and High findings only
3.0	May 2026	Added DEMO-01 RCE finding; renumbered all findings

Table of Contents

1. Executive Summary

About the Target

Demo SaaS Platform is a fictional B2B SaaS application used by companies to manage users, projects, invoices, reports, and role-based access across multiple tenants. The platform exposes a REST API consumed by a web frontend and supports three user roles: Admin, Manager, and User.

Engagement Summary

A manual Web & API security assessment was conducted against the Demo SaaS Platform. The assessment covered authentication, authorization, multi-tenant isolation, API data exposure, and server-side input handling — with specific focus on business logic flows, file processing pipelines, and how the platform enforces boundaries between tenants and privilege levels.

This was a manual assessment. Automated scanner output was not used as a primary source of findings. All findings were validated through manual exploitation with proof-of-concept evidence.

Key Findings

The assessment identified **5 findings** across two severity levels.

Severity	Count
Critical	2
High	3
Medium	0
Low	0
Informational	0
Total	**5**

Risk Summary for Stakeholders

Remote code execution is possible through the file import pipeline. The platform's CSV import feature passes user-controlled field values into a server-side command execution path without safe argument handling. An authenticated user can upload a crafted CSV that causes the backend import worker to execute arbitrary operating system commands. The worker process has access to internal configuration, environment variables, and tenant data being processed — meaning this vulnerability may allow full backend compromise and mass customer data exposure.

A complete tenant takeover is possible without exploiting any software vulnerability. The platform's user invitation flow does not verify that the accepting user is the person who was invited. Any authenticated user — from any tenant — can accept an Admin invitation intended

for someone else, gaining full administrative control over the victim organization. This requires no special technical skill and leaves minimal trace.

Customer data is accessible across account boundaries. An authenticated user from one company can retrieve project records and documents belonging to entirely separate companies by manipulating numeric object identifiers in API requests.

Role-based access controls exist only in the frontend. The API does not enforce the role hierarchy server-side. Any authenticated user can call Admin-only endpoints directly, including user management, role modification, and billing access.

Sensitive financial data is over-exposed in API responses. Invoice responses include internal fields that should never be visible to clients — payment method identifiers, internal pricing tiers, discount codes, and cost margin data.

Recommendation

DEMO-01 (RCE) and DEMO-02 (invitation takeover) are both Critical and require immediate remediation before continued use. DEMO-01 in particular should be treated as a potential exposure incident — any secrets accessible to the import worker during the testing window should be treated as compromised and rotated. The High findings reflect systemic gaps in authorization architecture that require middleware-level fixes, not per-endpoint patches.

---

2. Assessment Scope

In Scope

Asset	Details
Web Application	https://app.demosaas.io (fictional)
REST API	https://api.demosaas.io/v1 (fictional)
Admin Panel	https://admin.demosaas.io (fictional)
User Roles	Admin, Manager, User
Key Workflows	User management, project CRUD, invoice access, report download, tenant settings, user invitations, CSV file imports

Testing Environment

Field	Details
Environment	Staging
Test Accounts	Three accounts provided — one per role, across two fictional tenants
Testing Window	Business hours, Monday–Friday

Out of Scope

- Mobile applications
- Cloud infrastructure and network-level testing
- Third-party integrations (payment gateway, email provider)
- Social engineering or phishing
- DDoS or load testing
- Any system not listed above

---

3. Assessment Methodology

This assessment followed a manual, grey-box approach. Test accounts with pre-assigned roles were provided for each user tier across two separate tenant organizations, enabling cross-tenant authorization testing. Application source code was not provided.

Phases

Reconnaissance and Mapping Application structure, API endpoints, authentication flows, user roles, file upload and import workflows, and inter-tenant flows were mapped. HTTP traffic was captured and analysed to understand the full attack surface.

Authentication and Session Testing Login flows, token issuance, session lifecycle, token storage, and logout behaviour were tested.

Authorization Testing Horizontal and vertical access control was tested across all in-scope workflows. BOLA (Broken Object Level Authorization) and BFLA (Broken Function Level Authorization) patterns were specifically targeted given the multi-tenant, multi-role nature of the platform.

API Security Testing REST API endpoints were tested for insecure direct object references, response over-exposure, missing server-side authorization, and input handling issues.

File Upload and Import Testing File upload and CSV import workflows were tested for unsafe server-side processing, input validation gaps, and command injection vectors.

Business Logic Testing Workflows were tested for logic flaws — including cross-tenant operations, privilege abuse, and invitation flow bypasses — that would not be detected by automated scanners.

Reporting All findings were validated with proof-of-concept evidence. Critical and High findings were communicated immediately upon discovery, without waiting for the final report.

---

4. Assessment Limitations

This sample report is based on a fictional application and does not represent a complete security assessment of a real system. In real engagements, testing depth depends on agreed

scope, access level, environment stability, available test accounts, documentation, and testing window.

Security assessments are point-in-time activities. Findings reflect the state of the application during the testing period. New vulnerabilities may be introduced through subsequent code changes, dependency updates, or configuration drift.

No assessment can guarantee the identification of all vulnerabilities present in a system. This report covers the findings discovered within the agreed scope and testing window.

---

5. Risk Rating Methodology

Findings are rated using a combination of likelihood and business impact, informed by CVSS v3.1 base scores where applicable.

Severity	Description
Critical	Immediate, direct threat. Exploitable with low skill or effort. Severe impact: full backend compromise, tenant takeover, mass data breach, or arbitrary code execution. Immediate remediation required before continued production use.
High	Significant risk. Exploitable by an authenticated attacker with moderate effort. Serious impact to data confidentiality, integrity, or business operations. Remediate before the next production release.
Medium	Meaningful risk. May require specific conditions or chaining. Could expose sensitive data or compound other vulnerabilities. Remediate in the next planned cycle.
Low	Limited direct exploitability. Defence-in-depth gap or best practice deviation. Remediate as part of routine hardening.
Informational	No direct exploitability. Observation for security hygiene or attack surface reduction.

---

6. Summary of Findings

ID	Title	Severity	Business Area	Remediation Priority	Status
DEMO-01	RCE via File Import	Critical	File import / Backend worker	Immediate	Open
DEMO-02	Cross-Tenant Admin Takeover	Critical	Tenant isolation / Invitations	Immediate	Open
DEMO-03	BOLA Tenant Data Exposure	High	Tenant isolation / Project data	Immediate	Open
DEMO-04	Vertical Privilege Escalation	High	User management / Billing	Immediate	Open
DEMO-05		High			Open

ID	Title	Severity	Business Area	Remediation Priority	Status
	Invoice & Report Data Exposure		API exposure / Financial data	High priority / Next sprint	

7. OWASP Mapping

ID	Title	OWASP Top 10 2021	OWASP API Security 2023	ASVS
DEMO-01	RCE via File Import	A03 Injection / A05 Misconfiguration	API8:2023 Security Misconfiguration	V5.2 / V14.2
DEMO-02	Cross-Tenant Admin Takeover	A01 Broken Access Control	API5:2023 BFLA / API1:2023 BOLA	V4.1 / V4.2
DEMO-03	BOLA Tenant Data Exposure	A01 Broken Access Control	API1:2023 BOLA	V4.2
DEMO-04	Vertical Privilege Escalation	A01 Broken Access Control	API5:2023 BFLA	V4.1
DEMO-05	Invoice & Report Data Exposure	A01 Broken Access Control / A02	API3:2023 Broken Object Property Level Auth	V8.3 / V4.2

8. Detailed Findings

DEMO-01 · Remote Code Execution via Unsafe File Import Processing

Field	Details
ID	DEMO-01
Title	Remote Code Execution via Unsafe File Import Processing
Severity	Critical
Business Area	File import pipeline / Backend worker execution
OWASP Top 10 2021	A03 Injection / A05 Security Misconfiguration
OWASP API Security	API8:2023 Security Misconfiguration
ASVS	V5.2 Sanitization and Sandboxing / V14.2 Dependency
Affected Endpoint	`POST /api/v1/imports/csv`
Authentication Required	Yes
Status	Open

Description

The platform allows authenticated users to upload CSV files for bulk project and invoice imports. Uploaded files are processed by a backend worker service that reads each row and applies field values to internal processing logic.

During testing, it was identified that the import processor passes user-controlled CSV cell values into a server-side command execution path without adequate validation or safe argument handling. The worker uses a shell-interpreted execution model — equivalent to passing user input directly into a shell string — rather than parameterized argument arrays or safe library calls.

A malicious authenticated user can upload a crafted CSV file containing shell metacharacters in a field value. When the worker processes the file, the embedded expression is evaluated by the shell, causing arbitrary OS commands to execute under the worker process context.

This issue is especially severe because the import worker has access to internal application configuration, environment variables containing database connection strings and API keys, temporary files from other tenants' imports, and persistent storage paths.

Steps to Reproduce

1. Authenticate as `user@company-a.io` (User role) and capture the session token.
2. Create a CSV file with the following content:

```
project_name,budget,owner
"Q2 Migration","10000","$(whoami)"
```

1. Upload the file to the import endpoint:

```
POST /api/v1/imports/csv HTTP/1.1
Host: api.demosaas.io
Authorization: Bearer <Token-User>
Content-Type: multipart/form-data; boundary=----demo

------demo
Content-Disposition: form-data; name="file"; filename="projects.csv"
Content-Type: text/csv

project_name,budget,owner
"Q2 Migration","10000","$(whoami)"
------demo--
```

1. Observe the import worker log output (accessible via `GET /api/v1/imports/status` in the staging environment):

```
[import-worker] INFO  Received import job: job_cc7f3a1b
[import-worker] INFO  Processing row 1
[import-worker] DEBUG processing owner field: $(whoami)
```

```
[import-worker] DEBUG command output: app-worker
[import-worker] INFO  Row 1 imported successfully
```

Note: This is fictional, safe demonstration output. The command `whoami` identifies the running process user only. No destructive commands were executed during testing.

1. The worker log confirms that the shell expression `$(whoami)` was evaluated, returning `app-worker` — the identity of the backend worker process — rather than being treated as a literal string value.

Business Impact

An authenticated user with any role can execute arbitrary operating system commands on the backend import worker. Depending on the worker's runtime permissions and network access, successful exploitation may allow:

- Reading environment variables containing database credentials, internal API keys, and application secrets
- Reading temporary files from other tenants' in-progress imports
- Accessing internal filesystem paths including configuration files
- Lateral movement into internal services reachable from the worker's network context

In a multi-tenant SaaS environment, this represents a potential path to full backend compromise and mass customer data exposure. Any secrets accessible to the worker process during the testing window should be treated as potentially compromised.

Root Cause

- User-controlled CSV field values are passed directly into a shell command construction string without validation or escaping
- The import worker uses shell-interpreted execution (equivalent to `subprocess.run(f"process_field {value}", shell=True)` in Python or `exec("process " + value)` in Node.js) rather than safe parameterized argument arrays
- No allowlist validation is applied to CSV field values prior to processing
- The import worker process runs with broader OS-level permissions than are necessary for its function and has access to environment variables that contain production secrets

Remediation

- **Eliminate shell invocation with user input entirely.** Replace shell-based command execution with safe library calls or parameterized argument arrays. In Python: `subprocess.run(["process_field", value], shell=False)`. In Node.js: `execFile("process_field", [value])`. User-controlled input must never be interpolated into a shell string.
- **Apply strict server-side input validation.** Define an allowlist of permitted characters and patterns for each CSV field type (e.g. project name: alphanumeric, hyphens, spaces only). Reject any import file row that contains fields failing validation before processing begins.

- **Apply the principle of least privilege to the worker process.** The import worker should run under a dedicated low-privilege service account with no access to application secrets, database credentials, or other tenants' data paths.
- **Isolate import processing.** Run the worker in a container or sandbox with outbound network restrictions, a read-only filesystem where possible, and no access to the primary application environment variables.
- **Rotate exposed secrets.** Any credentials and API keys accessible to the worker process during the engagement window should be treated as compromised and rotated immediately.
- **Add regression tests.** Include automated tests that submit known command injection payloads (e.g. `$(id)`, `` `id` ``, `; id`) as CSV field values and assert they are processed as literal strings.
- **Add process execution monitoring.** Alert on unexpected child process spawning from the import worker service.

Verification Steps

1. Upload the same crafted CSV file (`$(whoami)` in the `owner` field) after the fix is deployed.
2. Confirm the import worker treats the value as a literal string — the field value stored should be `$(whoami)`, not the output of a command.
3. Confirm no OS command is spawned during processing (verify via process audit log or worker debug output).
4. Confirm worker logs show the field value was either sanitized, rejected as invalid, or stored verbatim without execution.
5. Confirm the worker service account no longer has access to application secrets or database credentials.
6. Confirm automated command injection regression tests pass in the CI test suite.

References

- OWASP Top 10 2021 — A03 Injection
- OWASP ASVS v4.0 — V5.2 Input Validation and Output Encoding
- OWASP Cheat Sheet Series — OS Command Injection Defense (see Appendix C)

DEMO-02 · Cross-Tenant Admin Takeover via Broken Invitation Flow

Field	Details
ID	DEMO-02
Title	Cross-Tenant Admin Takeover via Broken Invitation Flow
Severity	Critical
Business Area	Tenant isolation / User invitation / Admin access control
OWASP Top 10 2021	A01 Broken Access Control
OWASP API Security	API5:2023 Broken Function Level Authorization / API1:2023 BOLA

Field	Details
ASVS	V4.1 General Access Control Design / V4.2 Operation Level Access Control
Affected Endpoint	`POST /api/v1/invitations/{token}/accept`
Authentication Required	Yes
Status	Open

Description

The platform allows Admin users to invite new members to their tenant organization. When an invitation is created, a unique token is generated, associated with the invitee's email address and the invited role (e.g. Admin, Manager, User), and sent via email as a link. When the recipient clicks the link and authenticates, the acceptance endpoint processes the token and adds the authenticated user to the inviting organization with the specified role.

The critical flaw is that the `POST /api/v1/invitations/{token}/accept` endpoint validates that the token exists and has not expired, but **does not verify that the authenticated user's email address matches the email address recorded in the invitation**. The token is bound to a role and a target organization, but not to a specific identity.

This means any authenticated user — including users from completely unrelated tenants — can accept an invitation intended for someone else, gaining the invited role (including Admin) within the victim organization.

Steps to Reproduce

1. In a staging environment with two tenants — `company-a.io` and `company-b.io` — authenticate as `attacker@company-b.io` and capture the session token (`Token-B`).
2. From the `company-a.io` Admin panel, invite `newadmin@company-a.io` for the Admin role. The system generates an invitation token (e.g. `inv_a4f8b2c9d1e7f6a3`) and sends the link to `newadmin@company-a.io` .
3. The attacker obtains the invitation token — for example, through a forwarded email, by observing a link in shared communication, or by enumerating token values if the token format is predictable.
4. The attacker — still authenticated as `attacker@company-b.io` — sends the following request:

```
POST /api/v1/invitations/inv_a4f8b2c9d1e7f6a3/accept HTTP/1.1
Host: api.demosaas.io
Authorization: Bearer <Token-B>
Content-Type: application/json

{}
```

1. Observe the response:

```
HTTP/1.1 200 OK

{
  "status": "accepted",
  "organization": "company-a.io",
  "role": "admin",
  "user": "attacker@company-b.io"
}
```

1. The attacker is now an Admin member of `company-a.io`. They can create and delete users, access all projects and invoices, modify billing, download reports, and lock out legitimate administrators.

Business Impact

This vulnerability allows a complete takeover of any tenant organization by any authenticated user. The attacker gains Admin-level access — the highest privilege tier — without compromising any credential and without any interaction from the victim organization's users.

In a B2B SaaS environment, this represents the most severe class of multi-tenant isolation failure. A single attacker with a legitimate account on the platform can systematically target all organizations they can obtain invitation tokens for. The impact includes unauthorized access to all customer data within the compromised tenant, business disruption through account modification or lockout, and potential liability under data protection obligations.

Root Cause

The invitation acceptance endpoint does not enforce identity binding between the invitation record and the authenticated user. Token validation is limited to existence and expiry checks. The server does not compare the authenticated user's email against the `invitee_email` field stored in the invitation record at the point of acceptance.

This is a server-side authorization gap — the frontend correctly restricts access, but the API handler does not replicate that check.

Remediation

- **Bind invitations to identities.** On acceptance, compare the authenticated user's email address against the `invitee_email` stored in the invitation record. Reject the request with `403 Forbidden` if they do not match.
- **Single-use tokens.** Invalidate the invitation token immediately upon acceptance. Tokens should not be reusable.
- **Short expiry.** Enforce a short invitation validity window (e.g. 48–72 hours). Expired invitations should be automatically purged.
- **Audit logging.** Log all invitation acceptance events with the accepting user's identity, IP, and timestamp. Alert on acceptance from a mismatched email or unexpected domain.

- **Consider email re-verification.** For high-privilege role invitations (Admin), require the accepting user to verify ownership of the invited email address as part of the acceptance flow.

Verification Steps

1. Create an invitation for `newadmin@company-a.io` with Admin role.
2. Authenticate as `attacker@company-b.io` and attempt to accept the invitation token.
3. Confirm the API returns `403 Forbidden` with an error indicating identity mismatch.
4. Confirm `attacker@company-b.io` has not been added to `company-a.io`.
5. Accept the invitation as `newadmin@company-a.io` and confirm it succeeds correctly.
6. Attempt to reuse the same token as `newadmin@company-a.io` and confirm it returns a token-already-used error.

References

- OWASP API Security Top 10 2023 — API5:2023 Broken Function Level Authorization (see Appendix C)
- OWASP API Security Top 10 2023 — API1:2023 Broken Object Level Authorization (see Appendix C)
- OWASP ASVS v4.0 — V4.1 General Access Control Design

DEMO-03 · Broken Object Level Authorization Exposing Tenant Data

Field	Details
ID	DEMO-03
Title	Broken Object Level Authorization Exposing Tenant Data
Severity	High
Business Area	Tenant isolation / Project data access
OWASP Top 10 2021	A01 Broken Access Control
OWASP API Security	API1:2023 Broken Object Level Authorization
ASVS	V4.2 Operation Level Access Control
Affected Endpoints	`GET /api/v1/projects/{project_id}`, `GET /api/v1/projects/{project_id}/documents`, `GET /api/v1/projects/{project_id}/members`
Authentication Required	Yes
Status	Open

Description

The project API endpoints do not verify that the requested `project_id` belongs to the tenant of the authenticated user. The authorization check confirms only that a valid session token is present. Any authenticated user, regardless of which organization they belong to, can access project records from any other tenant by supplying an arbitrary numeric `project_id` in the request path.

This affects not only the core project record but the full resource tree beneath it: documents, member lists, and associated metadata are all accessible through the same pattern. Sequential integer identifiers make enumeration straightforward.

Steps to Reproduce

1. Authenticate as `user@company-b.io` and note the session token (`Token-B`). Note that `company-b.io` has no relationship with `company-a.io` .
2. Within `company-b.io` 's own account, observe that project IDs are sequential integers (e.g. the user's own project is ID `1042`).
3. Send the following request with `Token-B` :

```
GET /api/v1/projects/1199 HTTP/1.1
Host: api.demosaas.io
Authorization: Bearer <Token-B>
```

1. Observe the response returns a complete project record belonging to `company-a.io` :

```
HTTP/1.1 200 OK

{
  "id": 1199,
  "name": "Q2 Infrastructure Rollout",
  "tenant": "company-a.io",
  "owner": "cto@company-a.io",
  "status": "active",
  "members": ["alice@company-a.io", "bob@company-a.io"],
  "created_at": "2026-01-14T09:00:00Z"
}
```

1. The same pattern applies to documents and member lists under the same project ID.

Business Impact

Any authenticated user on the platform can read all project records, document attachments, and member lists belonging to other customers. In a B2B SaaS environment serving multiple competing companies, this represents a direct breach of the platform's data isolation guarantee. Depending on the data stored, this may also constitute a breach of contractual confidentiality obligations or applicable data protection regulations.

Root Cause

The endpoint handler queries the project by ID without joining against the authenticated user's tenant context. The query is effectively `SELECT * FROM projects WHERE id = ?` rather than `SELECT * FROM projects WHERE id = ? AND tenant_id = ?`. Authorization is treated as authentication — a valid token is sufficient to access any object.

Remediation

- **Tenant-scope all object queries server-side.** Every query retrieving a tenant-owned resource must include the authenticated user's `tenant_id` as a filter condition. An object that exists but belongs to a different tenant should return `404 Not Found` to avoid confirming the object's existence.
- **Apply the pattern consistently.** Audit all API endpoints that accept object identifiers in the path or query string. The same flaw is likely present on invoice, report, and settings endpoints.
- **Replace sequential integer IDs.** Use UUIDs for externally exposed object identifiers. This does not fix the authorization flaw but reduces the ease of enumeration.
- **Centralize authorization in middleware.** Implement a consistent authorization middleware that enforces tenant scoping before the handler executes.

Verification Steps

1. Authenticate as a user in Tenant B. Note your own project IDs.
2. Request a project ID from Tenant A's range.
3. Confirm the API returns `404 Not Found` rather than the project data.
4. Confirm the fix applies consistently across `/documents` and `/members` sub-resources.
5. Confirm write and delete operations on cross-tenant objects are also rejected.

References

- OWASP API Security Top 10 2023 — API1:2023 Broken Object Level Authorization (see Appendix C)
- OWASP ASVS v4.0 — V4.2 Operation Level Access Control
- OWASP Web Security Testing Guide — Testing for Insecure Direct Object References

---

DEMO-04 · Vertical Privilege Escalation via Missing Server-Side Role Validation

Field	Details
ID	DEMO-04
Title	Vertical Privilege Escalation via Missing Server-Side Role Validation
Severity	High
Business Area	User management / Billing access
OWASP Top 10 2021	A01 Broken Access Control

Field	Details
OWASP API Security	API5:2023 Broken Function Level Authorization
ASVS	V4.1 General Access Control Design
Affected Endpoints	`POST /api/v1/admin/users`, `DELETE /api/v1/admin/users/{id}`, `PATCH /api/v1/admin/users/{id}/role`, `GET /api/v1/billing/invoices`, `GET /api/v1/billing/subscription`
Authentication Required	Yes
Status	Open

Description

The platform enforces role-based access control in the frontend UI: User-role accounts do not see the Admin menu, user management pages, or billing sections. However, the underlying API endpoints that serve these features perform no server-side role check. Any authenticated user — regardless of their assigned role — can call Admin-only endpoints directly and receive successful responses.

Affected operations include creating new users, deleting existing accounts, modifying user roles, and reading all billing and subscription data. The frontend restriction is entirely bypassable by any user who can send an HTTP request.

Steps to Reproduce

1. Authenticate as `user@company-a.io` (User role) and capture the session token (`Token-User`).
2. Confirm in the UI that the user has no access to Admin or Billing sections.
3. Send the following requests directly, bypassing the frontend:

Access billing records:

```
GET /api/v1/billing/invoices HTTP/1.1
Host: api.demosaas.io
Authorization: Bearer <Token-User>
```

```
HTTP/1.1 200 OK

{
  "invoices": [
    { "id": "inv_001", "amount": 12000, "status": "paid", "date": "2026-04-01" },
    { "id": "inv_002", "amount": 12000, "status": "pending", "date": "2026-05-01" }
  ]
}
```

Escalate own role to Admin:

```
PATCH /api/v1/admin/users/usr_8821/role HTTP/1.1
Host: api.demosaas.io
Authorization: Bearer <Token-User>
Content-Type: application/json

{ "role": "admin" }
```

```
HTTP/1.1 200 OK

{ "id": "usr_8821", "role": "admin" }
```

Business Impact

Any authenticated user can promote themselves to Admin, create backdoor accounts with elevated privileges, read all billing and subscription data, and delete other users' accounts. This effectively nullifies the entire access control model. The risk is especially acute in organizations with external contractors or trial users who hold User-role accounts — any such account can become a full Admin without any action from a legitimate administrator.

Root Cause

Role checks are implemented only in the frontend routing layer. The API handlers verify that a valid session token is present (authentication) but do not check the token's associated role against the required privilege level for the endpoint (authorization). Authentication and authorization have not been separated server-side.

Remediation

- **Enforce role checks server-side on every privileged endpoint.** A valid session token confirms who the user is; a separate role check is required to confirm what they are allowed to do.
- **Implement a centralized authorization middleware** that maps each endpoint to a required minimum role and rejects requests that do not meet it.
- **Default to deny.** Apply the principle of least privilege — explicitly grant access to specific roles rather than relying on frontend restrictions.
- **Audit all `/admin/` and `/billing/` routes** for missing role guards. Treat the endpoints above as a sample, not an exhaustive list.

Verification Steps

1. Authenticate as a User-role account.
2. Attempt `GET /api/v1/billing/invoices` — confirm `403 Forbidden`.
3. Attempt `POST /api/v1/admin/users` — confirm `403 Forbidden`.
4. Attempt `PATCH /api/v1/admin/users/{own_id}/role` — confirm `403 Forbidden`.
5. Authenticate as an Admin-role account and confirm all three operations succeed.
6. Confirm role checks are applied at the API layer, not just the UI.

References

- OWASP API Security Top 10 2023 — API5:2023 Broken Function Level Authorization (see Appendix C)
- OWASP ASVS v4.0 — V4.1 General Access Control Design
- OWASP Testing Guide — Testing for Privilege Escalation

DEMO-05 · Sensitive Invoice and Report Data Exposure via Broken Object Property Authorization

Field	Details
ID	DEMO-05
Title	Sensitive Invoice and Report Data Exposure via Broken Object Property Authorization
Severity	High
Business Area	API data exposure / Invoice and report data
OWASP Top 10 2021	A01 Broken Access Control / A02 Cryptographic Failures
OWASP API Security	API3:2023 Broken Object Property Level Authorization
ASVS	V8.3 Sensitive Private Data / V4.2 Operation Level Access Control
Affected Endpoints	`GET /api/v1/invoices/{invoice_id}`, `GET /api/v1/reports/{report_id}/download`
Authentication Required	Yes
Status	Open

Description

Two related issues were identified on the invoice and report endpoints that together expose sensitive financial and operational data to users who should not have access to it.

Issue A — Invoice response over-exposes internal fields. The `GET /api/v1/invoices/{invoice_id}` endpoint returns the full internal invoice object rather than a filtered client-facing representation. Fields returned include Stripe payment method identifiers, internal pricing tiers, discount codes, cost margin data, and tax configuration — none of which have any legitimate purpose in a client-facing API response.

Issue B — Report download accessible to User-role accounts. The `GET /api/v1/reports/{report_id}/download` endpoint performs no role check. User-role accounts can download full report PDFs that should be restricted to Admin and Manager roles only.

Steps to Reproduce

Issue A — Invoice field over-exposure:

```
GET /api/v1/invoices/inv_001 HTTP/1.1
Host: api.demosaas.io
Authorization: Bearer <Token-User>
```

```
HTTP/1.1 200 OK

{
  "id": "inv_001",
  "amount": 12000,
  "currency": "INR",
  "status": "paid",
  "date": "2026-04-01",
  "stripe_payment_method_id": "pm_1OqXzZLkdIwHu7ix4J2xNYRp",
  "stripe_customer_id": "cus_PqR7tUvWxYz123",
  "internal_pricing_tier": "growth_v2_discounted",
  "discount_code": "PARTNER2026",
  "discount_pct": 22,
  "cost_margin_pct": 61,
  "tax_exemption_status": "exempt_b2b",
  "account_manager_notes": "Strategic account — do not auto-churn"
}
```

Issue B — Report download by User-role:

```
GET /api/v1/reports/rpt_112/download HTTP/1.1
Host: api.demosaas.io
Authorization: Bearer <Token-User>
```

The API streams back the full report PDF, which includes Admin-visible aggregate data, all-user activity logs, and financial summaries.

Business Impact

Issue A: Exposure of Stripe payment method identifiers, internal pricing tiers, and discount codes allows customers to infer commercial terms negotiated with other accounts if identifiers are guessable. Account manager notes and margin data expose confidential commercial strategy. These fields should never appear in any client-facing API response.

Issue B: Report download access by User-role accounts exposes data the platform explicitly restricts in its own UI — including activity logs for all users in the organization and financial summaries that are Admin-only views.

Root Cause

Issue A: The invoice handler serializes the internal ORM model directly to JSON without a response DTO or field allowlist. Any field on the internal model is returned to the client.

Issue B: The report download handler checks for a valid session token but not for the required role. The restriction is enforced only in the frontend UI.

Remediation

Issue A:

- Define an explicit serialization schema for the invoice API response. Include only fields the frontend requires: `id`, `amount`, `currency`, `status`, `date`, and formatted line items.
- Do not serialize ORM models directly to API responses. Use a response DTO or serializer with an explicit field allowlist.

Issue B:

- Add a server-side role check to the report download endpoint. User-role accounts requesting the download should receive `403 Forbidden`.
- Apply the same centralized authorization middleware described in DEMO-04.

Verification Steps

1. Authenticate as a User-role account and request `GET /api/v1/invoices/{id}`.
2. Confirm the response does not include `stripe_payment_method_id`, `stripe_customer_id`, `internal_pricing_tier`, `discount_code`, `cost_margin_pct`, or `account_manager_notes`.
3. Authenticate as a User-role account and request `GET /api/v1/reports/{id}/download`.
4. Confirm the API returns `403 Forbidden`.
5. Authenticate as an Admin-role account and confirm the report download succeeds.

References

- OWASP API Security Top 10 2023 — API3:2023 Broken Object Property Level Authorization (see Appendix C)
- OWASP ASVS v4.0 — V8.3 Sensitive Private Data
- OWASP ASVS v4.0 — V4.2 Operation Level Access Control

---

9. Retest Summary

This section is completed after the client has performed remediation and requested a retest.

ID	Title	Severity	Remediation Status	Retest Date	Result
DEMO-01	RCE via File Import	Critical	Pending	—	—
DEMO-02	Cross-Tenant Admin Takeover	Critical	Pending	—	—
DEMO-03	BOLA Tenant Data Exposure	High	Pending	—	—
DEMO-04	Vertical Privilege Escalation	High	Pending	—	—

ID	Title	Severity	Remediation Status	Retest Date	Result
DEMO-05	Invoice & Report Data Exposure	High	Pending	—	—

Retest findings will be documented here after the client confirms remediation. Each finding will be marked Resolved, Partially Resolved, or Unresolved with supporting evidence.

---

10. Appendix

Appendix A — Tools Used

Tool	Purpose
Burp Suite Professional	HTTP traffic interception, manual testing, API analysis, file upload testing
Browser DevTools	Client-side analysis, storage inspection, network traffic review
curl / httpie	Targeted API request crafting and response analysis
jwt.io	JWT structure and claims inspection

This was a manual assessment. Automated scanning was not used as a primary testing method.

Appendix B — Glossary

Term	Definition
BOLA	Broken Object Level Authorization — an attacker accesses objects they are not authorized to access by manipulating identifiers
BFLA	Broken Function Level Authorization — an attacker invokes privileged functions by calling API endpoints without appropriate role enforcement
RCE	Remote Code Execution — a vulnerability that allows an attacker to execute arbitrary code on a target system
OS Command Injection	A form of injection where user-controlled input is passed to a shell command interpreter, allowing execution of arbitrary OS commands
Multi-tenant	An architecture where a single application instance serves multiple independent customers (tenants), with strict data isolation between them
IDOR	Insecure Direct Object Reference — a specific implementation pattern of BOLA where internal object identifiers are directly exposed in requests
ASVS	OWASP Application Security Verification Standard — a framework of security requirements for web applications
CVSS	Common Vulnerability Scoring System — a standardized framework for rating the severity of security vulnerabilities

Term	Definition
DTO	Data Transfer Object — an object used to carry data between processes or layers, typically used to filter and shape API response payloads
ORM	Object-Relational Mapper — a software layer that maps database rows to in-memory objects; serializing an ORM object directly to an API response is a common cause of field over-exposure

Appendix C — References

- OWASP Top 10 2021: https://owasp.org/Top10/
- OWASP Top 10 2021 — A03 Injection: https://owasp.org/Top10/A03_2021-Injection/
- OWASP API Security Top 10 2023: https://owasp.org/API-Security/
- OWASP API1:2023 BOLA: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/
- OWASP API3:2023 Broken Object Property Level Authorization: https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/
- OWASP API5:2023 BFLA: https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/
- OWASP ASVS v4.0: https://owasp.org/www-project-application-security-verification-standard/
- OWASP Testing Guide v4.2: https://owasp.org/www-project-web-security-testing-guide/
- OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/
- OWASP OS Command Injection Defense Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html

---

This is a sample report. All application names, URLs, findings, and data are fictional and for illustration purposes only. Actual engagement reports are delivered under a mutual non-disclosure agreement.

RedSpire Security
Founded by Harshit Shah · Cybersecurity Consultant
harshit@redspiresecurity.com
+91 94622 55272
https://redspiresecurity.com/
https://www.linkedin.com/in/harshit-shah-699a54127/